

TASK1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 500
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 200

/*
 * Parse command and handle escape sequences.
 *
 * Examples:
 * Hello\ World      -> Hello World
 * Hello\ $USER      -> Hello $USER
 * file\&name        -> file&name
 * hello\*world      -> hello*world
 */
int parse_command(char *input,
                  char tokens[][MAX_TOKEN_LENGTH])
{
    int i = 0;
    int token_count = 0;

    while (input[i] != '\0')
    {
        /* Skip normal spaces */
        while (isspace((unsigned char)input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        char token[MAX_TOKEN_LENGTH];
        int j = 0;

        /*
         * Read one token.
         */
        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]))
        {
            /*
             * Escape sequence
             */
            if (input[i] == '\\')
            {
                i++;

                /*
                 * If '\' is at the end,
                 * report an error.
                 */
                if (input[i] == '\0')
                {
                    printf("\nERROR: Escape character at end of input.\n");
                    return -1;
                }

                /*
                 * Preserve the character after '\'
                 * without interpreting it.
                 */
                if (j < MAX_TOKEN_LENGTH - 1)
                {
                    token[j++] = input[i];
                }
            }
            else
            {
                token[j++] = input[i];
            }
            i++;
        }

        tokens[token_count++] = token;
    }
}
```

```

printf("Total tokens: %d\n", count);

for (int i = 0; i < count; i++)
{
    printf("Token %d: [%s]\n",
           i + 1,
           tokens[i]);
}

printf("\n=====\\n");
}

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];

    printf("=====\\n");
    printf("      ESCAPE SEQUENCE PARSER IN C\\n");
    printf("=====\\n");

    printf("\\nSupported escape examples:\\n");
    printf("  Hello\\ World    -> Hello World\\n");
    printf("  Hello\\$USER      -> Hello$USER\\n");
    printf("  file\\&name       -> file&name\\n");
    printf("  hello\\*world     -> hello*world\\n");
    printf("  question\\?      -> question?\\n");
    printf("  slash\\\\\\test    -> slash\\\\test\\n");

    printf("\\nEnter a command:\\n> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("ERROR: Unable to read input.\\n");
        return 1;
    }

    /*
     * Remove newline.
     */
    input[strcspn(input, "\\n")] = '\\0';

    /*
     * Check empty input.
     */
    if (strlen(input) == 0)
    {
        printf("ERROR: Empty input.\\n");
        return 1;
    }

    /*
     * Parse input.
     */
    int count = parse_command(input, tokens);

    if (count == -1)
    {
        printf("\\nParsing failed.\\n");
        return 1;
    }

    /*
     * Validate output.
     */
    validate_output(tokens, count);

    printf("\\nProgram completed successfully.\\n");

    return 0;
}

```

```

Program completed successfully.
satwika@SATWIKAG:~$ nano escape_parser.c
satwika@SATWIKAG:~$ gcc escape_parser.c -o escape_parser
satwika@SATWIKAG:~$ ./escape_parser
=====
                ESCAPE SEQUENCE PARSER IN C
=====

Supported escape examples:
Hello\ World      -> Hello World
Hello\ $USER      -> Hello $USER
file\&name        -> file&name
hello\*world      -> hello*world
question\?        -> question?
slash\\test       -> slash\test

Enter a command:
> echo Hello\ World

===== PARSER OUTPUT =====
Parsing successful!
Total tokens: 2

Token 1: [echo]
Token 2: [Hello World]

=====

Program completed successfully.
satwika@SATWIKAG:~$ |

```

TASK2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    pid_t pid;
    int status;

    printf("=====\n");
    printf("    PROCESS LAUNCHER USING FORK & EXEC\n");
    printf("=====\n");

    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());

    printf("\nCreating child process using fork()...\n");

    /*
     * Create child process
     */
    pid = fork();

    /*
     * Check fork error
     */
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    /*
     * Child process
     */
    if (pid == 0)
    {
        printf("\n----- CHILD PROCESS ----- \n");

        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: ls -l\n");

        /*
         * Execute ls command.
         *
         * execlp() searches for the program
         * in the PATH environment variable.
         */
        execlp("ls", "ls", "-l", NULL);

        /*
         * This line executes ONLY if exec fails.
         */
        perror("exec failed");

        exit(1);
    }

    /*
     * Parent process
     */
    else
    {
        printf("\n----- PARENT PROCESS ----- \n");

        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }
}

```

```

    */
    perror("exec failed");
    exit(1);
}

/*
 * Parent process
 */
else
{
    printf("\n----- PARENT PROCESS ----- \n");

    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("\nParent waiting for child process...\n");

    /*
     * Wait for child to finish
     */
    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        return 1;
    }

    /*
     * Check child exit status
     */
    if (WIFEXITED(status))
    {
        printf("\nChild process completed.\n");
        printf("Child exit status: %d\n",
               WEXITSTATUS(status));
    }
    else
    {
        printf("\nChild terminated abnormally.\n");
    }

    printf("\nParent process continues.\n");
    printf("Parent process completed successfully.\n");
}

return 0;
}

```

```

satwika@SATWIKAG:~$ nano process_launcher.c
satwika@SATWIKAG:~$ gcc process_launcher.c -o process_launcher
satwika@SATWIKAG:~$ ./process_launcher
=====
PROCESS LAUNCHER USING FORK & EXEC
=====

Parent Process
Parent PID: 1498

Creating child process using fork()...

----- PARENT PROCESS -----
Parent PID: 1498
Child PID: 1499

Parent waiting for child process...

----- CHILD PROCESS -----
Child PID: 1499
Parent PID: 1498

Executing command: ls -l
total 408
-rw-r--r-- 1 satwika satwika 96 Aug 20 13:55 File1.txt
-rwxr-xr-x 1 satwika satwika 16440 Sep 15 08:08 double_quotes
-rw-r--r-- 1 satwika satwika 5852 Sep 15 08:08 double_quotes.c
-rwxr-xr-x 1 satwika satwika 16360 Sep 15 08:45 escape_parser
-rw-r--r-- 1 satwika satwika 4138 Sep 15 08:44 escape_parser.c
-rw-r--r-- 1 satwika satwika 1116 Aug 20 12:16 fork.c
-rwxr-xr-x 1 satwika satwika 16480 Aug 20 12:25 fork2
-rw-r--r-- 1 satwika satwika 1224 Aug 20 12:24 fork2.c
-rwxr-xr-x 1 satwika satwika 16272 Aug 20 12:16 fork_exec
-rw-r--r-- 1 satwika satwika 684 Aug 1 04:16 fork_exec.c
-rwxr-xr-x 1 satwika satwika 16272 Aug 1 04:16 frok_exec
-rwxr-xr-x 1 satwika satwika 15968 Jul 25 04:08 helloworld
-rwxr-xr-x 1 satwika satwika 16608 Aug 11 06:49 myShell
-rw-r--r-- 1 satwika satwika 2061 Aug 11 06:49 myShell.c
-rwxr-xr-x 1 satwika satwika 16352 Aug 20 13:04 p2t1
-rw-r--r-- 1 satwika satwika 1236 Aug 20 13:01 p2t1.c
-rwxr-xr-x 1 satwika satwika 16256 Aug 20 13:22 p3t1
-rw-r--r-- 1 satwika satwika 1069 Aug 20 13:23 p3t1.c
-rwxr-xr-x 1 satwika satwika 16256 Aug 20 13:27 p3t2
-rw-r--r-- 1 satwika satwika 1117 Aug 20 13:26 p3t2.c
-rw-r--r-- 1 satwika satwika 891 Aug 18 15:16 practical3.c
-rwxr-xr-x 1 satwika satwika 16256 Aug 8 02:55 process
-rw-r--r-- 1 satwika satwika 1063 Aug 8 02:54 process.c
-rwxr-xr-x 1 satwika satwika 16368 Sep 15 08:50 process_launcher
-rw-r--r-- 1 satwika satwika 2171 Sep 15 08:50 process_launcher.c
-rwxr-xr-x 1 satwika satwika 16368 Sep 15 08:03 quote_parser
-rw-r--r-- 1 satwika satwika 3287 Sep 15 08:03 quote_parser.c
-rwxr-xr-x 1 satwika satwika 16224 Aug 20 13:49 s3t1
-rw-r--r-- 1 satwika satwika 702 Aug 20 13:49 s3t1.c
-rwxr-xr-x 1 satwika satwika 16216 Aug 20 13:56 s3t2
-rw-r--r-- 1 satwika satwika 1110 Aug 20 13:54 s3t2.c
-rwxr-xr-x 1 satwika satwika 16312 Aug 20 14:05 s3t3
-rw-r--r-- 1 satwika satwika 1592 Aug 20 14:04 s3t3.c
-rwxr-xr-x 1 satwika satwika 16448 Aug 22 03:35 s4t1
-rw-r--r-- 1 satwika satwika 3423 Aug 22 03:34 s4t1.c
-rwxr-xr-x 1 satwika satwika 16088 Aug 8 02:57 states
-rw-r--r-- 1 satwika satwika 503 Aug 8 02:57 states.c
-rwxr-xr-x 1 satwika satwika 16224 Aug 4 07:01 task1
-rw-r--r-- 1 satwika satwika 418 Aug 4 06:59 task1.c

Child process completed.
Child exit status: 0

Parent process continues.
Parent process completed successfully.
satwika@SATWIKAG:~$ |

```